



TP6

“Fonction d’ordre supérieur, système de typage data”

Exercice 1

Dans cet exercice, on représente un polynôme $a_n x^n + a_{n-1} x^{n-1} + \dots + a_0 x^0$ par une liste de ses coefficients $a_0, \dots, a_{n-1}, a_n$ en ordre croissant des puissances. Par exemple, le polynôme $x^3 + 5x + 2$ est représenté par la liste $[2, 5, 0, 1]$.

1. En utilisant la notation **type**, créer les types **Puissance**, **Coeff**, **Valeur** et **Polynome** où **Puissance** est la puissance d’un terme du polynôme représenté par entier primitif, **Coeff** est le coefficient d’un terme du polynôme représenté par un entier de grande taille, **Valeur** est la valeur qu’on peut associer à une variable ou un résultat d’un calcul d’un polynôme, et **Polynome** est une liste de coefficients.
2. En utilisant les types créés, définir les fonctions suivantes sur des polynômes dans cette représentation :

- (a) Évaluation pour une valeur de x :

```
evaluerPoly :: Polynome -> Valeur -> Valeur
```

- (b) Addition de deux polynômes :

```
plusPoly :: Polynome -> Polynome -> Polynome
```

- (c) Multiplication de deux polynômes :

```
-- une fonction auxlière pour multiplier une constante fois un polynôme  
multParCst :: Coeff -> Polynome -> Polynome  
-- utilisant les fonctions précédentes  
multPoly :: Polynome -> Polynome -> Polynome  
-- penser à rajouter une position tout à gauche dans le polynôme pour  
-- tenir compte p.ex. une addition avec  $(3X^3 + 1) + (X^2 + 1)$   
-- plusPoly [3,0,0,1] + [1,0,1] ==> plusPoly [3,0,0,1] 0:[1,0,1]
```

- (d) Calcul de la dérivée d’un polynôme :

```
-- une fonction pour calculer la dérivée d'un polynôme  
deriveePoly :: Polynome -> Polynome
```

- (e) Impression en chaîne de caractères explicite d’un polynôme à partir d’une liste de coefficients. Définir une fonction impressionPolyZF avec une ZF-expression. Vous pouvez définir d’autres fonctions auxiliaires (**afficherCoeff** et **afficherPuissance**) afin d’afficher de façon individuelle le coefficient et la variable avec sa puissance pour chaque terme du polynôme. Vous pouvez aussi utiliser la fonction standard **show** pour convertir tout type en chaîne de caractères.

```
afficherCoeff :: Coeff -> Puissance -> String
afficherPuissance :: Puissance -> String
impressionPolyZF, :: Polynome -> String

*Main> impressionPolyZF [1,0,5,2]
"1 + 5x^2 + 2x^3"
```

Exercice 2

Soit le type suivant :

```
data Temp = Celsius Float | Fahrenheit Float | Kelvin Float
```

Sachant que $F = 32 + 9/5C$, et $K = C + 273$,

1. écrire des fonctions de conversions de température

```
toCelsius, toFahrenheit, toKelvin :: Temp -> Temp
```

2. écrire des fonctions de comparaison entre températures

```
eqTemp, infTemp :: Temp -> Temp -> Bool
```

Exercice 3

On considère une liste de voyageurs décrits par leur nom, leur prénom, et leur âge. Pour cet exercice, la liste est donnée comme suit :

```
voyageurs = [ Voyageur "Métivier" "Alice" 30
              , Voyageur "Dubois" "Patrice" 62
              , Voyageur "Roche" "Charlie" 30
              , Voyageur "Petit" "Rose" 23
              , Voyageur "Fontaine" "Eve" 28
              , Voyageur "Guillaume" "Julien" 2
            ]
```

L'objectif est de regrouper les voyageurs se situant dans la même plage d'âge. On considère les tranches d'âge suivantes :

- 0 - 3 ans
- 4 - 29 ans
- 30 - 59 ans
- 60 ans et plus

1. En utilisant la notation **data**, définir un type de données **Voyageur** en Haskell avec les champs mentionnés en-dessus.

2. Définir une fonction `groupVoyageurs` qui renvoie une liste de voyageurs groupés avec les tranches d'âge mentionnées précédemment.

```
groupVoyageurs :: [Voyageur] -> ([Voyageur], [Voyageur], [Voyageur], [Voyageur])
```

Exercice 4

Dans cet exercice, nous allons implémenter la logique propositionnelle dans Haskell. Pour y arriver, nous créerons notre propre type `Formule` pour avoir une représentation de formules propositionnelles. Ce nouveau type regroupe deux valeurs **Vrai** et **Faux** avec trois constructeurs dont le premier `Non` qui prend une seule formule pour renvoyer la négation, le deuxième `Et` qui prend deux formules pour envoyer le et logique, le dernier `Ou` qui prend deux formules pour envoyer le ou logique.

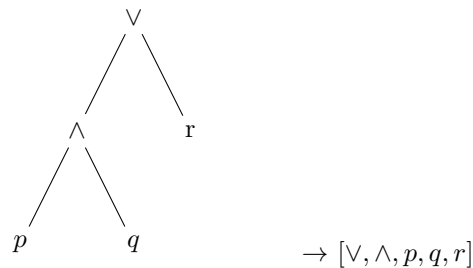
1. Définir ce nouveau type dans votre code Haskell en utilisant la notation `data`.

```
data Formule = ?
```

2. Définir des fonctions (neutres et absorbants) pour évaluer les valeurs **Vrai**, **Faux** et **Négation** des deux valeurs.

```
eval :: Formule -> Formule
```

3. L'évaluation d'une formule logique peut-être représentée par un arbre. Par exemple, la formule $(p \wedge q) \vee (r)$ est représentée par :



Définir une fonction `evaluerRec` qui consiste à appliquer l'opérateur logique (**et** ou **ou**) récursivement aux résultats de l'évaluation du fils gauche et du fils droit. Cette fonction ne tient pas compte des fonctions (neutres et absorbants) définies dans 2 :

```
eval (Et Vrai Vrai) = Vrai -- le cas unique vrai pour et logique
eval (Et _ _)      = Faux -- le reste des cas est toujours faux (absorbant)
-- ...
```

Cette évaluation doit consister à arrêter l'évaluation dès qu'on rencontre **Faux** pour un **et** logique et **Vrai** pour un **ou** logique.

```
evaluerRec :: Formule -> Formule
```
